Ripple *Radar*

Hitachi Hack the Highway | TAMU Datathon 2026

An event-driven operations control tower for Wacky Wheels Co.

***Orbit's** email was two lines long: the Huntsville cameras would arrive Thursday, not Monday. Behind it sat six builds with no parts, 54 customer orders running late, 26 dealers to warn and ten loaner cars to hand out fairly.*

Ripple Radar finds all of that, and it is careful about who gets to decide what happens next.

How we built it

We made one decision early on: AI reads the message. Code does everything else. Qwen2.5 3B, running locally, pulls out the details someone is reporting. It doesn't decide which production orders move, which customers hear about it, or which vehicles stop selling. Python does, using the company's records and the handbook.

A Coordinator routes each incident to three specialists:

Supply Chain	Production scheduling and transfers
Dealer & Customer Care	Dealer notices and loaners
Quality	Defective lots and affected vehicles

They trace each disruption through a knowledge graph of 14 node types and 17,772 nodes, showing how suppliers, parts, builds, dealers and customers connect. Our traces record the evidence path behind every action.

How it works

Read AI › **Verify** Code › **Trace** Graph › **Plan** Agents › **Check** RuleBook › **Execute** Enterprise

Unverified information or an invalid plan goes to review. Only approved actions reach the enterprise server.

How it performed

15/15	768/768	0
Event readings matched	Enterprise calls accepted	Differences from a validator written separately from the planner

We also tested how 240 generated messages were read: 221 matched the expected reading and 19 went to review, with no wrong actionable reading. These are results from the supplied simulator and our own testing, not the hidden grader.

Why AI doesn't decide

During E-09, Qwen mistook "Carrier", the trucking company, for a supplier. No supplier has that name, so our code took the real one from the shipment record. The model can misunderstand a sentence, and the rest of the system still has to work with real records.

Guardrails

Values the model returns must appear in the message and match real records. If one fails, the model gets one retry, then a person takes over. Before any call is sent, Python verifies the relationships in the graph, checks all ten handbook policies and rehearses the calls on a separate enterprise copy. Unconfirmed reports go to review, and noise produces no calls at all. If a write might have succeeded but the server doesn't respond, we stop rather than send it again.

What we'd still improve

The handbook's transfer rule can be read more than one way. Under another reasonable reading, 5 of our 768 calls get different dates, and we'd want that clarified. We'd also try two small models cross-checking each reading, and what-if planning for future disruptions.

E-09 impact in the graph

STEP	EXAMPLE	REACHED
SHIPMENT	**SH-0256, Orbit Optics**	1
PART	**CAM-360, camera kit**	1
MODEL	**SUMMIT**	2
BUILD	**PO-0436, Huntsville**	6
ORDER	**CO-02567**	54
DEALER	**D25, Tupelo, MS**	26

Model: Qwen2.5-3B-Instruct (Qwen Team, 2024; arXiv:2412.15115), run locally in Ollama (`qwen2.5:3b`, `Q4_K_M`), Qwen Research License. No hosted model or API key.